JonesAuto Database Management System

CPSC 3660 Course Project
Gurman Basran
April 2026

1 Stage 1: Overall Design

JonesAuto is a used car dealership in Lethbridge that buys vehicles at auction, fixes them up, and sells them to customers. This project builds a database system to track everything from purchases and repairs to sales, warranties, and payments.

1.1 System Goals

The system needs to handle the full lifecycle of a used car: buying it at auction, recording any repairs, selling it to a customer, setting up warranties, and tracking monthly payments. Beyond just storing data, it should also give the dealership useful reports like which cars are still on the lot, how much profit each sale made, and which customers are behind on payments.

Everything runs through a web interface built with PHP and MySQL. Employees don't need any special training; they just fill out forms and click through reports.

1.2 Input Forms

Form	Purpose
Purchase Form	Record a vehicle bought at auction with repair problems
Sale Form	Sell a vehicle to a customer and record financing
Warranty Form	Add warranty coverage items to a completed sale
Payment Form	Record monthly payments from customers
Customer Form	Add, edit, or soft-delete customers
Employee Form	Add, edit, or soft-delete employees

The Customer and Employee forms support editing and deleting records. Deletes are soft-deletes: the record stays in the database but gets flagged as inactive with `is_active = 0`. This keeps referential integrity intact since other tables still point to those records.

1.3 Output Reports

Report	Purpose
Current Inventory	Shows all available vehicles with cost info
Sales Report	Lists all sales with profit calculations
Payment History	Shows payment records for a specific customer
Repair Summary	Lists all repairs grouped by vehicle
Warranty Report	Shows warranty status and expiration dates
Late Payments	Flags customers who've paid late

1.4 System Users

The system is meant for dealership staff: buyers who go to auctions and purchase vehicles, salespeople who sell them to customers, and managers who want to see the big picture through reports. There's no login or role-based access control; anyone at the dealership can access any form or report. In a real deployment you'd add authentication, but for this project it isn't required.

1.5 Initial Design Assumptions

- Every vehicle enters the system through a purchase (auction buy). JonesAuto doesn't accept trade-ins or consignment vehicles.
- A vehicle can only be sold once. After it's sold, its status changes to "sold" and it disappears from inventory.
- Repairs are always tied to a purchase, not to a sale or warranty claim. They represent work done before the car hits the lot.
- Warranties are tied to sales, not directly to vehicles. A customer can add multiple warranty items to the same sale.
- Payments track both the due date and the actual paid date, so the system can calculate how many days late a payment was.
- All tables (except operations_log) have an `is_active` flag for soft-delete support. Records are never physically removed from the database.
- Employees can be buyers, salespeople, or both. The role field determines which dropdowns they show up in.
- Customer credit info (late payments, average days late) is stored directly on the customer record for quick lookups during sales.

2 Stage 2: ER Diagram and Database Schema

The ER diagram uses Chen notation, which represents entities as rectangles, relationships as diamonds, and attributes as ellipses. Lines between entities and relationships show cardinality: a "1" means one record on that side, and an "N" means many. Double lines indicate total participation, meaning every record on that side must participate in the relationship. Single lines mean partial participation, where some records might not be involved.

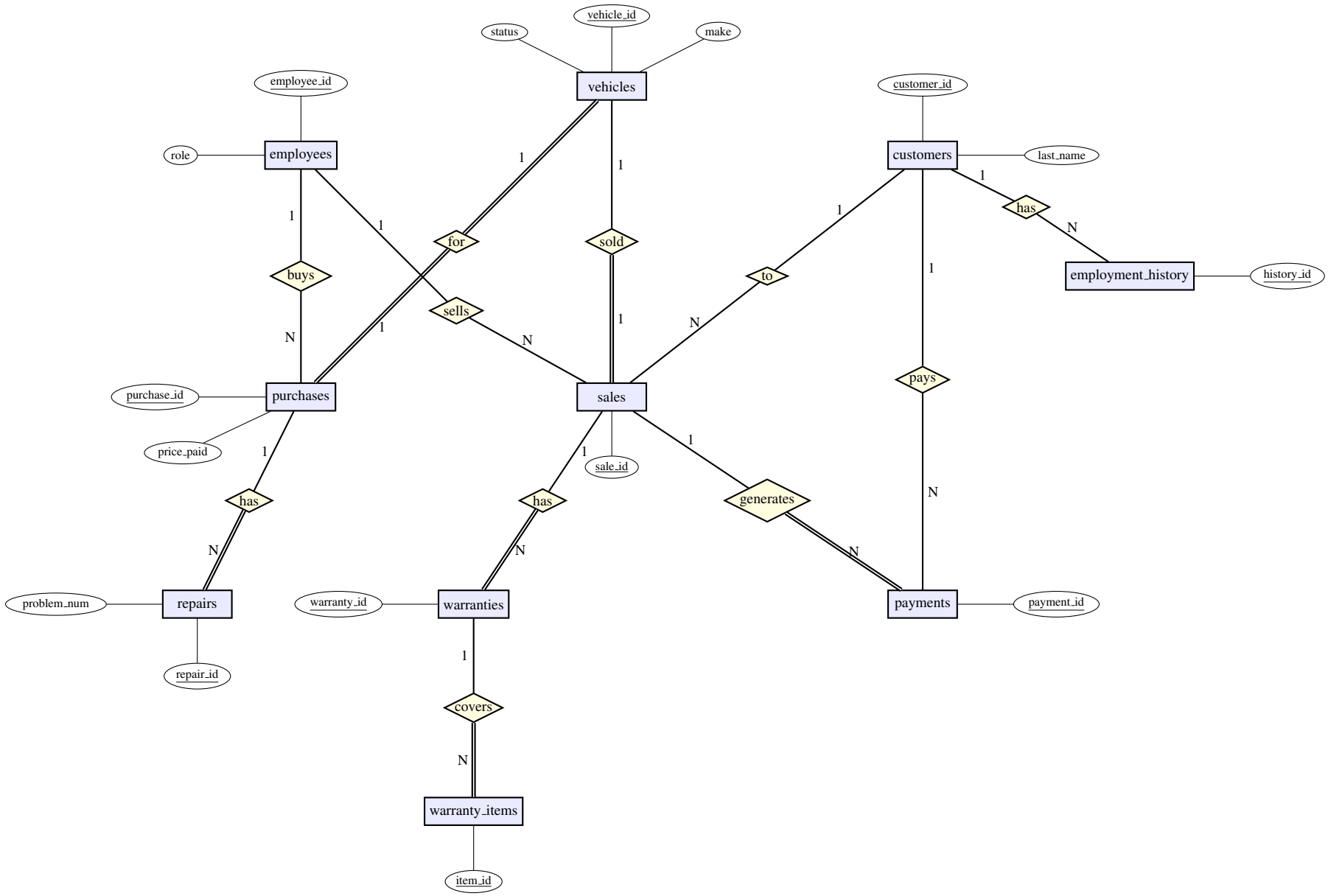


Figure 1: ER Diagram (Chen Notation)

2.1 Entities and Attributes

Entity	Attributes	Primary Key
employees	employee_id, first_name, last_name, phone, role, is_active	employee_id
vehicles	vehicle_id, make, model, year, color, miles, condition_desc, book_price, style, interior_color, status, is_active	vehicle_id
purchases	purchase_id, vehicle_id (FK), employee_id (FK), purchase_date, location, seller_dealer, is_auction, price_paid, is_active	purchase_id
repairs	repair_id, purchase_id (FK), problem_num, description, est_cost, actual_cost, is_active	repair_id
customers	customer_id, first_name, last_name, phone, address, city, state, zip, gender, dob, num_late_payments, avg_days_late, is_active	customer_id
employment_history	history_id, customer_id (FK), employer, title, supervisor_phone, employer_address, start_date, is_active	history_id
sales	sale_id, vehicle_id (FK), customer_id (FK), employee_id (FK), sale_date, total_due, down_payment, financed_amount, sale_price, commission, is_active	sale_id
warranties	warranty_id, sale_id (FK), vehicle_id (FK), customer_id (FK), employee_id (FK), warranty_sale_date, total_cost, monthly_cost, is_active	warranty_id
warranty_items	item_id, warranty_id (FK), warranty_type, start_date, length_months, cost, deductible, items_covered, is_active	item_id
payments	payment_id, customer_id (FK), sale_id (FK), payment_date, due_date, paid_date, amount, bank_account, is_active	payment_id
operations_log	log_id, table_name, record_id, operation, op_date	log_id

Every table except operations_log includes an `is_active` column. This is how soft-deletes work: instead of running a `DELETE` statement, the system sets `is_active = 0` and logs the operation. All queries filter on `is_active = 1` so deleted records don't show up in forms or reports, but they're still in the database if someone needs to look them up later.

2.2 Relationships

1. An employee **buys** vehicles at auction, creating purchases (1:N, partial on both sides).

2. Each purchase is **for** exactly one vehicle, and each vehicle comes from exactly one purchase (1:1, total participation on both sides).
3. An employee **sells** vehicles, creating sales (1:N, partial on both sides).
4. A vehicle is **sold** in at most one sale (1:1, partial on vehicle side, total on sale side).
5. A sale goes **to** a customer (N:1, partial on both sides).
6. A purchase **has** zero or more repairs (1:N, partial on purchase, total on repair).
7. A sale **has** zero or more warranties (1:N, partial on sale, total on warranty).
8. A warranty **covers** one or more warranty items (1:N, partial on warranty, total on items).
9. A sale **generates** payments (1:N, partial on sale, total on payment).
10. A customer **pays** their payments (1:N, partial on both sides).
11. A customer **has** employment history records (1:N, partial on both sides).

2.3 Relational Schema

The relational schema diagram below shows how foreign keys connect the tables. Arrows point from the child table to the parent table, with the FK column labeled on each arrow.

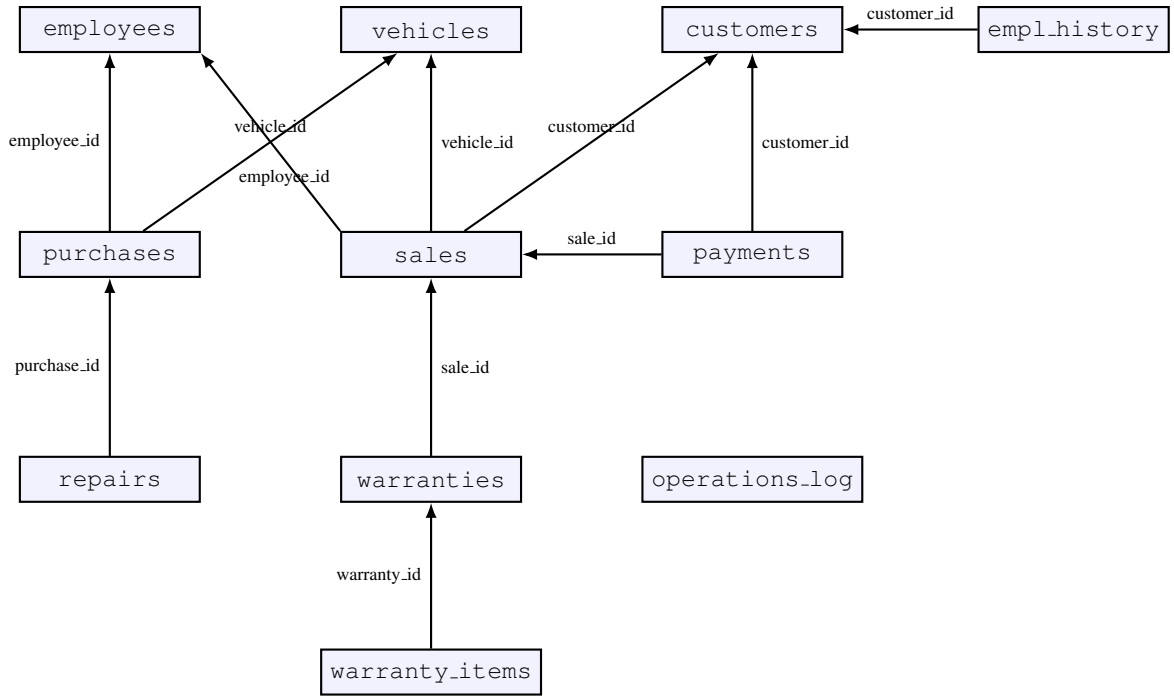


Figure 2: Relational Schema Diagram

The operations_log table doesn't have foreign key relationships to other tables. It just stores a record of what table was affected, which record ID, and what operation happened (insert, update, or delete). It's a standalone audit trail.

2.4 Normalization

All tables are in Third Normal Form (3NF). Each non-key column depends on the primary key, the whole primary key, and nothing but the primary key.

There are two intentional denormalizations:

Customer credit stats: The customers table stores `num_late_payments` and `avg_days_late` directly, even though this data could be calculated from the payments table. I did this because the sale form shows customer credit info in a popup when you select a customer, and recalculating it from payments every time would be slow. The process scripts update these fields whenever a new payment is recorded.

Warranty redundant FKs: The warranties table has `vehicle_id`, `customer_id`, and `employee_id` columns even though these can all be derived through the sale. This makes it easier to query warranty data without always joining through the sales table. It's a tradeoff: a bit of redundancy for simpler and faster queries.

3 Stage 3: Forms and Reports

3.1 Input Forms

3.1.1 Purchase Form

The purchase form is where everything starts. When JonesAuto buys a vehicle at auction, the buyer fills out the purchase details (date, location, seller, price paid) and the vehicle info (make, model, year, color, miles, condition, book price, style, interior color). The form also has a repair section at the bottom with a dynamic "Add Repair" button that adds more rows via JavaScript. Each repair gets a problem number, description, estimated cost, and actual cost. Below the form there's a table showing the 10 most recent purchases for reference.

3.1.2 Sale Form

The sale form is the most complex one. You pick a vehicle from a dropdown of available cars, and a green popup appears showing the vehicle's book price, mileage, condition, what we paid for it, repair costs, and total investment. This helps the salesperson set a good price. You fill in the sale price, total due, down payment, and financed amount, then pick a salesperson and set the commission.

For the customer, you can either pick an existing customer or create a new one with a radio button toggle. When you select an existing customer, a yellow info box shows their phone number, how many late payments they have, and their average days late. This is the credit check info the salesperson needs.

At the bottom there's an employment history section with a dynamic "Add Employer" button, since the assignment requires tracking where customers work for credit purposes.

3.1.3 Warranty Form

The warranty form is tied to a sale. You select a sale from a dropdown (which shows the customer name, vehicle, and date), and the page reloads to show the warranty form for that sale. This page-reload approach means the form always has the correct sale context. You enter the warranty date, total cost, and monthly cost, then add warranty items with type (Drive-Train, Exterior, Interior, Electrical), start date, length in months, cost, deductible, and items covered.

After you submit, the page reloads and shows the warranty you just added in an "Existing Warranties" section at the bottom. This makes it easy to add multiple warranties to the same sale without losing track of what's already been entered.

3.1.4 Payment Form

The payment form starts by selecting a customer. The page reloads and shows that customer's stats: gender, date of birth, number of late payments, and average days late. Then you pick which sale the payment is for (since a customer might have bought multiple cars), and fill in the payment date, due date, paid date, amount, and bank account.

Below the form there's a payment history table for that customer, with late payments highlighted in red. This gives the employee immediate context about whether this customer tends to pay on time.

3.1.5 Customer Form

The customer form handles creating, editing, and deleting customers. When you visit it fresh, you get a blank form to add a new customer with fields for name, phone, address, city, state, zip, gender, and date of birth. Below the form there's a table of all existing customers with Edit and Delete links.

Clicking Edit loads that customer's data into the form fields so you can update them. Clicking Delete does a soft-delete by setting `is_active = 0` and logging the operation. When you're editing a customer, the page also shows their employment history and a form to add new employment records.

3.1.6 Employee Form

The employee form works the same way as the customer form but with fewer fields: first name, last name, phone, and role (salesperson, buyer, or both). It also lists existing employees at the bottom with Edit and Delete links. Deletes are soft-deletes, same as customers.

3.2 Output Reports

3.2.1 Current Inventory

The inventory report shows all vehicles currently available on the lot. It includes the make, model, year, color, miles, condition, book price, what we paid, repair costs, and the total investment (price

paid + repairs). There's a filter at the top to search by make. Each row has a "Sell" action link that takes you directly to the sale form with that vehicle pre-selected.

3.2.2 Sales Report

The sales report lists every completed sale with the vehicle info, customer name, salesperson, sale price, cost basis (price paid + repairs), and calculated profit. There's a date range filter at the top. Summary totals at the bottom show total revenue, total costs, and total profit across all displayed sales. Each row has an "Add Warranty" action link to jump straight to the warranty form for that sale.

3.2.3 Payment History

The payment history report shows all payment records with customer name, vehicle, due date, paid date, amount, and whether the payment was on time or late. Late payments are highlighted in red. There's a customer filter at the top. Each row includes a "Record Payment" link to go to the payment form for that customer.

3.2.4 Repair Summary

The repair summary groups all repairs by vehicle, showing the problem description, estimated cost, actual cost, and whether it went over budget. This helps management see which vehicles cost more to fix than expected.

3.2.5 Warranty Report

The warranty report shows all active warranties with customer name, vehicle, warranty type, start date, expiration date, and how many days are left. Warranties expiring within 90 days are highlighted so the dealership can follow up with customers about renewals.

3.2.6 Late Payments

The late payments report shows only customers who have at least one late payment. It displays the customer name, phone, total number of payments, how many were late, and their average days late. Each row has a "Record Payment" link. This report helps the dealership identify problem customers quickly.

4 Stage 4: Business Queries

The system includes 12 business queries that the dealership can run from a dropdown menu on the queries page. The first five are simple queries, and the remaining seven are more advanced.

4.1 Query 1: What cars do we have on the lot right now?

This is a straightforward `SELECT` with a `WHERE` clause filtering on two conditions: the vehicle status has to be "available" and the record has to be active. The `ORDER BY` sorts results alphabetically by make and model so it's easy to scan through the list.

```
SELECT * FROM vehicles
WHERE status = 'available' AND is_active = 1
ORDER BY make, model
```

4.2 Query 2: Show me all our customers

Another simple `SELECT` that pulls basic customer info. It uses a column list instead of `SELECT *` to only grab what's needed for a quick overview, and sorts by last name.

```
SELECT customer_id, first_name, last_name, phone, city, state
FROM customers
WHERE is_active = 1
ORDER BY last_name
```

4.3 Query 3: Do we have any [make] cars in stock?

This query takes user input for the make and filters inventory by it. It combines the make filter with status and active checks in the `WHERE` clause. The user types in a make like "Toyota" and the query finds matching available vehicles.

```
SELECT * FROM vehicles
WHERE make = '[make]'
AND status = 'available' AND is_active = 1
```

4.4 Query 4: Who are our employees?

Simple query that lists all active employees, sorted first by role and then by last name within each role. This groups buyers, salespeople, and dual-role employees together.

```
SELECT * FROM employees
WHERE is_active = 1
ORDER BY role, last_name
```

4.5 Query 5: How many cars did we buy this month?

Uses the aggregate functions `COUNT` and `SUM` to get a quick summary. The `MONTH()` and `YEAR()` functions extract the current month and year from the purchase date and compare them to `CURDATE()` so the query always returns the current month's numbers.

```
SELECT COUNT(*) as cars_bought, SUM(price_paid) as total_spent
FROM purchases
WHERE MONTH(purchase_date) = MONTH(CURDATE())
```

```
AND YEAR(purchase_date) = YEAR(CURDATE())
AND is_active = 1
```

4.6 Query 6: Which cars did we buy below book price?

This is the first advanced query. It uses a `JOIN` between vehicles and purchases to compare the book price against what we actually paid. The calculated column `savings` shows the difference, and `ORDER BY savings DESC` puts the best deals at the top.

```
SELECT v.make, v.model, v.year, v.book_price, p.price_paid,
       (v.book_price - p.price_paid) as savings
FROM vehicles v
JOIN purchases p ON v.vehicle_id = p.vehicle_id
WHERE p.price_paid < v.book_price AND v.is_active = 1
ORDER BY savings DESC
```

4.7 Query 7: What is our profit on each sale?

This is probably the most complex query. It joins sales, vehicles, and purchases, then uses a correlated subquery to sum up repair costs for each vehicle. The profit is calculated as sale price minus price paid minus repair costs. The subquery with `COALESCE` handles cases where a vehicle has no repairs by defaulting to zero.

```
SELECT v.make, v.model, v.year, s.sale_price, p.price_paid,
       (SELECT COALESCE(SUM(r.actual_cost),0) FROM repairs r
        WHERE r.purchase_id = p.purchase_id) as repair_costs,
       (s.sale_price - p.price_paid -
        (SELECT COALESCE(SUM(r.actual_cost),0) FROM repairs r
         WHERE r.purchase_id = p.purchase_id)) as profit
FROM sales s
JOIN vehicles v ON s.vehicle_id = v.vehicle_id
JOIN purchases p ON v.vehicle_id = p.vehicle_id
WHERE s.is_active = 1
ORDER BY profit DESC
```

4.8 Query 8: Which customers have late payments?

Uses a four-table `JOIN` chain (payments to customers, to sales, to vehicles) and the `DATEDIFF` function to calculate how many days late each payment was. The `WHERE` clause filters to only payments where the paid date is after the due date. `DISTINCT` prevents duplicate rows when a customer has multiple late payments for the same vehicle.

```
SELECT DISTINCT CONCAT(c.first_name, ' ', c.last_name) as customer_name,
       c.phone, v.make, v.model, v.year,
       DATEDIFF(pay.paid_date, pay.due_date) as days_late
FROM payments pay
```

```
JOIN customers c ON pay.customer_id = c.customer_id
JOIN sales s ON pay.sale_id = s.sale_id
JOIN vehicles v ON s.vehicle_id = v.vehicle_id
WHERE pay.paid_date > pay.due_date AND pay.is_active = 1
ORDER BY days_late DESC
```

4.9 Query 9: Who is our top salesperson?

Joins sales and employees, then uses `GROUP BY` on the employee to aggregate their sales stats. `COUNT` gives the number of sales, `SUM` gives total revenue and total commission. Sorting by `num_sales DESC` puts the top performer at the top.

```
SELECT CONCAT(e.first_name, ' ', e.last_name) as salesperson,
       COUNT(s.sale_id) as num_sales,
       SUM(s.sale_price) as total_revenue,
       SUM(s.commission) as total_commission
FROM sales s
JOIN employees e ON s.employee_id = e.employee_id
WHERE s.is_active = 1
GROUP BY e.employee_id
ORDER BY num_sales DESC
```

4.10 Query 10: Which warranties expire within 90 days?

This query joins four tables to connect warranty items back to customers and vehicles. It uses `DATE_ADD` with `INTERVAL` to calculate expiration dates from the start date and length, then `DATEDIFF` to figure out how many days are left. The `BETWEEN` clause in the `WHERE` filters to warranties expiring in the next 90 days.

```
SELECT CONCAT(c.first_name, ' ', c.last_name) as customer_name,
       v.make, v.model, v.year,
       wi.warranty_type, wi.start_date,
       DATE_ADD(wi.start_date, INTERVAL wi.length_months MONTH)
           as expiry_date,
       DATEDIFF(DATE_ADD(wi.start_date,
           INTERVAL wi.length_months MONTH),
           CURDATE()) as days_remaining
FROM warranty_items wi
JOIN warranties w ON wi.warranty_id = w.warranty_id
JOIN vehicles v ON w.vehicle_id = v.vehicle_id
JOIN customers c ON w.customer_id = c.customer_id
WHERE DATE_ADD(wi.start_date, INTERVAL wi.length_months MONTH)
      BETWEEN CURDATE() AND DATE_ADD(CURDATE(), INTERVAL 90 DAY)
      AND wi.is_active = 1
ORDER BY expiry_date
```

4.11 Query 11: Where did repair costs go over budget?

Joins repairs, purchases, and vehicles, then uses `GROUP BY` on the purchase to sum up estimated and actual costs per vehicle. The `HAVING` clause filters to only show vehicles where actual costs exceeded estimates. This is different from `WHERE` because it filters on aggregated values after grouping.

```
SELECT v.make, v.model, v.year,
       SUM(r.est_cost) as estimated_total,
       SUM(r.actual_cost) as actual_total,
       (SUM(r.actual_cost) - SUM(r.est_cost)) as over_budget
FROM repairs r
JOIN purchases p ON r.purchase_id = p.purchase_id
JOIN vehicles v ON p.vehicle_id = v.vehicle_id
WHERE r.is_active = 1
GROUP BY p.purchase_id
HAVING SUM(r.actual_cost) > SUM(r.est_cost)
ORDER BY over_budget DESC
```

4.12 Query 12: Which customers bought more than one car?

Joins customers and sales, groups by customer, and uses `HAVING COUNT > 1` to find repeat buyers. This combines `GROUP BY`, `COUNT`, `SUM`, and `HAVING` in one query to identify the dealership's best customers.

```
SELECT CONCAT(c.first_name, ' ', c.last_name) as customer_name,
       c.phone,
       COUNT(s.sale_id) as cars_bought,
       SUM(s.sale_price) as total_spent
FROM customers c
JOIN sales s ON c.customer_id = s.customer_id
WHERE s.is_active = 1
GROUP BY c.customer_id
HAVING COUNT(s.sale_id) > 1
ORDER BY cars_bought DESC
```

5 Assumptions and Design Decisions

1. **Soft-delete everywhere:** Records are never physically deleted. Every table (except operations_log) has an `is_active` flag, and all queries filter on `is_active = 1`. This keeps foreign key references intact and lets the system recover deleted records if needed.

2. **Operations log:** Every insert, update, and delete is logged to the `operations_log` table with the table name, record ID, and operation type. This creates an audit trail without adding complexity to the main tables.

3. **One purchase per vehicle:** The system assumes each vehicle enters inventory through exactly one purchase. There's no mechanism for a vehicle to be returned and resold.

4. **Credit stats on customer:** Instead of calculating late payment stats on the fly, the customer record stores `num_late_payments` and `avg_days_late` directly. This is updated whenever a payment is recorded. It's a denormalization, but it makes the sale form's credit check popup fast.

5. **Employee roles:** Employees can be buyers, salespeople, or both. The system uses this role to filter which employees appear in the buyer dropdown (purchase form) vs. the salesperson dropdown (sale form). Someone with the "both" role shows up in both.

6. **Warranty structure:** Warranties have two levels: the warranty record (tied to a sale, with total and monthly cost) and warranty items (individual coverage types with their own start dates, lengths, costs, and deductibles). This handles cases where a customer buys multiple types of coverage.

7. **Vehicle status:** Vehicles have a status field that changes from "available" to "sold" when a sale is recorded. Only available vehicles show up in the inventory report and sale form dropdown.

8. **Employment history for credit:** The system tracks customer employment history (employer, title, supervisor phone, address, start date) for credit evaluation purposes. This can be entered during a sale or added later through the customer form.

9. **Views for common queries:** The database includes several views (`v_active_inventory`, `v_sales_profit`, `v_late_customers`, `v_warranty_status`, `v_operations`, `v_deleted_rec` to simplify frequently used queries and make the codebase cleaner.

6 Challenges

1. **MySQL socket permissions:** When I first set up MySQL on the Linux environment, the PHP scripts couldn't connect because the socket file had wrong permissions. The error messages weren't very helpful; it just said "connection refused." I had to figure out that the socket at `/var/run/mysqld/mysqld.sock` needed the right ownership and permissions for the web server user to access it.

2. **Re-running the setup script:** The `db_setup.sql` script creates tables, views, and seed data. But if you run it twice, it fails because the tables already exist. I had to add `DROP TABLE IF EXISTS` and `DROP VIEW IF EXISTS` statements at the top, and they have to be in the right order because of foreign key constraints. Child tables need to be dropped before parent tables.

3. **ONLY_FULL_GROUP_BY mode:** MySQL's strict mode requires every non-aggregated column in a `SELECT` to appear in the `GROUP BY` clause. Some of my queries were grouping by just the primary key but selecting other columns from the same table, which threw errors. I had to either add all selected columns to the `GROUP BY` or restructure the queries.

4. **Warranty form page-reload:** The warranty form needed to show existing warranties for the selected sale, but the sale selection has to happen before the form renders. I solved this by making the sale dropdown trigger a page reload with the sale ID in the URL. The PHP code checks for this parameter and conditionally renders the warranty form and existing warranty list. It's not as smooth as AJAX would be, but it works and keeps the code simple.
5. **Soft-delete consistency:** Adding `is_active` to every table meant I had to remember to filter on it in every single query, form dropdown, and report. Missing it in even one place would show deleted records. I also had to make sure the operations_log entry gets created alongside the soft-delete update, which meant two queries for every delete operation.

7 What I Learned

Going into this project, I understood the basics of relational databases from class lectures, but actually building one from scratch taught me a lot more. Drawing the ER diagram forced me to think carefully about which entities are really separate things vs. just attributes of something else. For example, warranty items could've been columns on the warranty table, but breaking them into their own entity made the design way more flexible since one warranty can cover multiple types of things.

Normalization made a lot more sense after I had to deal with it in practice. When I first designed the tables, I had customer address info duplicated across sales and the customer record. Moving it to just the customer table and referencing it through foreign keys cleaned things up. I also learned that sometimes you want to intentionally break normalization for performance, like storing the late payment count on the customer record so the sale form doesn't have to run an expensive aggregate query every time.

Writing PHP with mysqli was a good exercise in understanding how web applications talk to databases. The process scripts that handle form submissions taught me about the full cycle: receive POST data, validate it, build an INSERT or UPDATE query, execute it, and redirect back with a success or error message. It's not glamorous code, but it's how a lot of real-world applications still work.

The ER diagram was probably the most useful artifact from the whole project. Being able to look at it and trace relationships helped me write queries and debug issues. When a query wasn't returning the right data, I could look at the diagram and see which tables I needed to join and through which foreign keys.

Working with foreign keys and referential integrity taught me why soft-deletes matter. Early on I tried using actual `DELETE` statements and kept hitting foreign key constraint errors because other tables still referenced the deleted records. Switching to soft-deletes with `is_active` solved all of that.

8 Future Improvements

If I had more time, I'd add user authentication so different employees only see what's relevant to their role, maybe restrict buyers from accessing the sale form and vice versa. I'd also add pagination to the reports since they'll get unwieldy once there are hundreds of records. Search and filtering could be better too; right now only the inventory report has a filter. It would be nice to add AJAX to the forms so things like the warranty form don't need a full page reload when you select a sale. And the queries page could use some charts or graphs instead of just tables, maybe a bar chart showing monthly sales or a line graph of revenue over time.

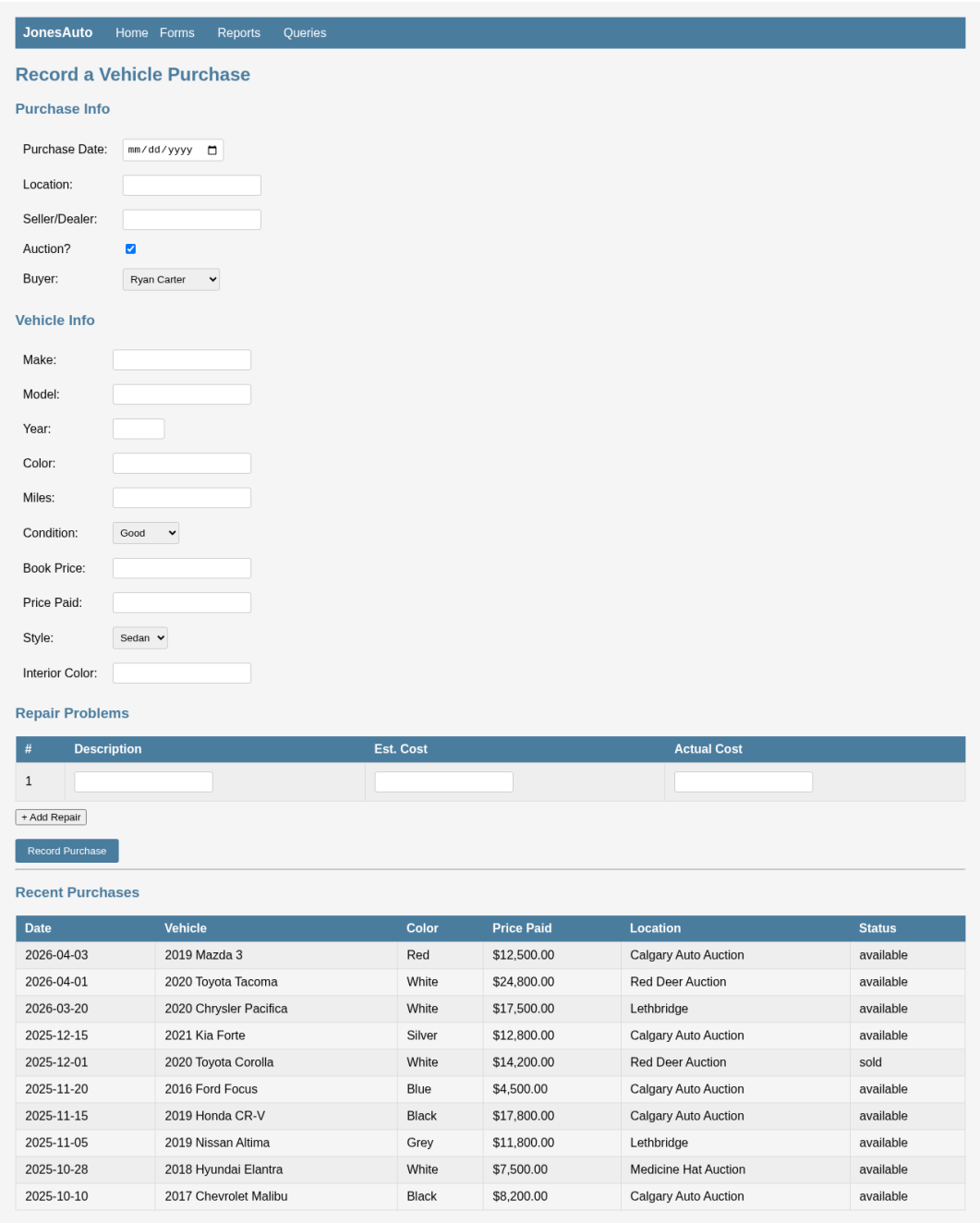


Figure 3: Purchase Form

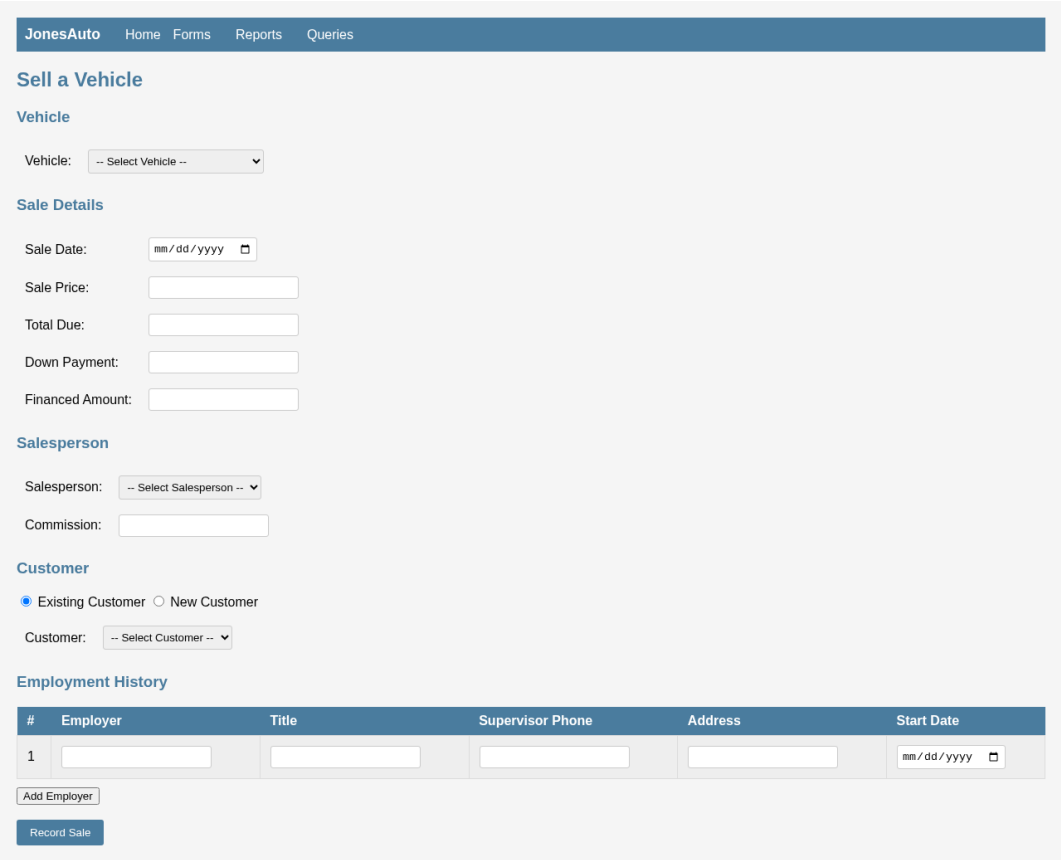


Figure 4: Sale Form

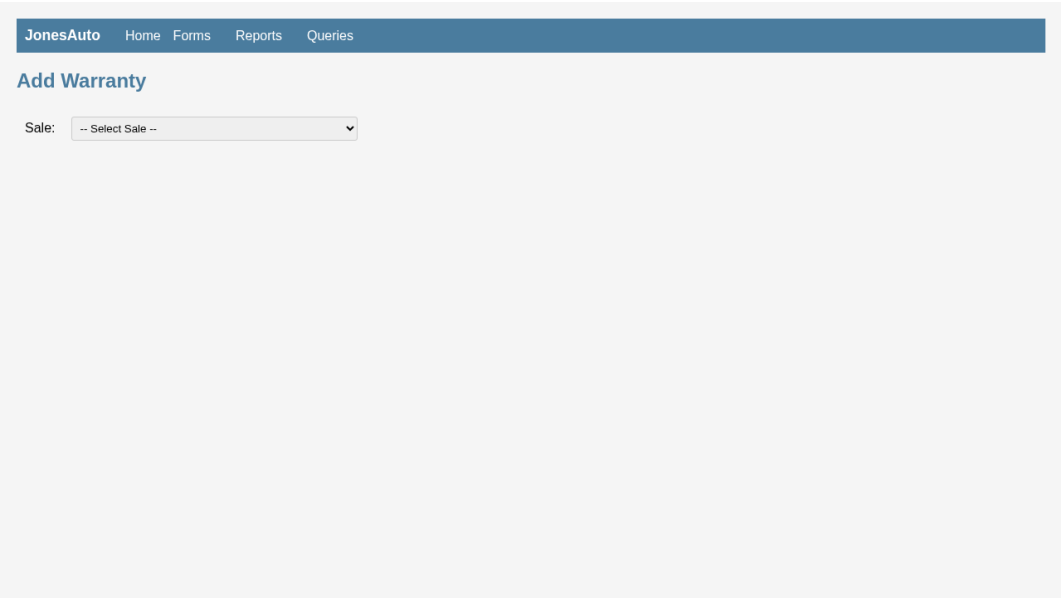


Figure 5: Warranty Form

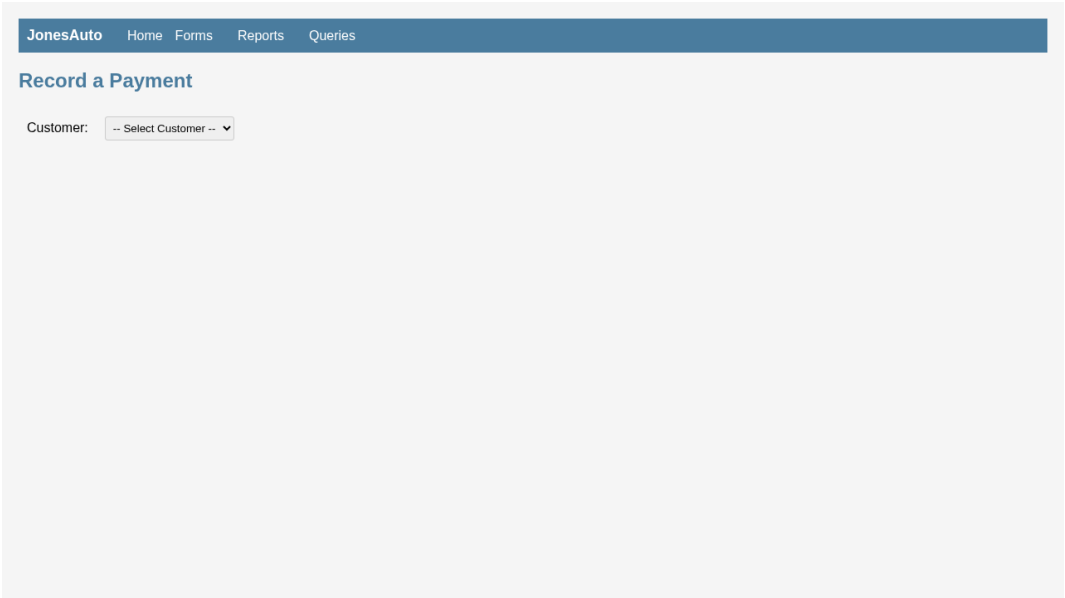
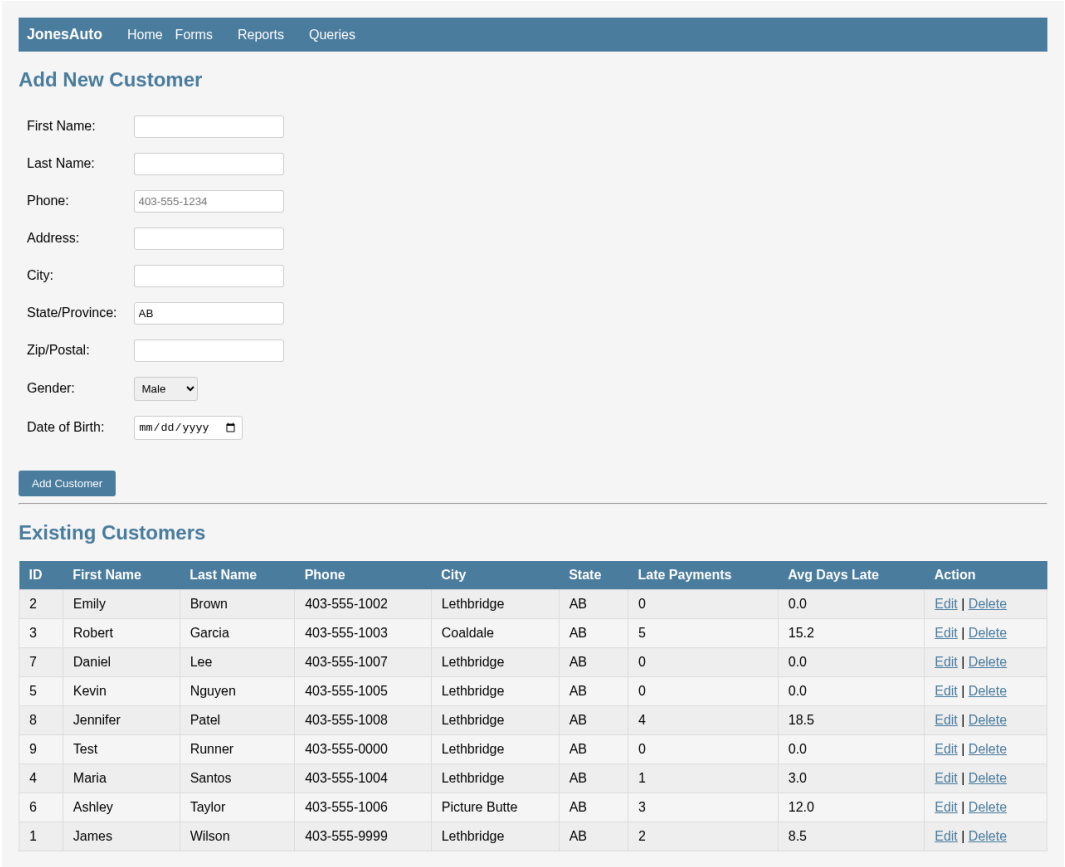


Figure 6: Payment Form



ID	First Name	Last Name	Phone	City	State	Late Payments	Avg Days Late	Action
2	Emily	Brown	403-555-1002	Lethbridge	AB	0	0.0	Edit | Delete
3	Robert	Garcia	403-555-1003	Coaldale	AB	5	15.2	Edit | Delete
7	Daniel	Lee	403-555-1007	Lethbridge	AB	0	0.0	Edit | Delete
5	Kevin	Nguyen	403-555-1005	Lethbridge	AB	0	0.0	Edit | Delete
8	Jennifer	Patel	403-555-1008	Lethbridge	AB	4	18.5	Edit | Delete
9	Test	Runner	403-555-0000	Lethbridge	AB	0	0.0	Edit | Delete
4	Maria	Santos	403-555-1004	Lethbridge	AB	1	3.0	Edit | Delete
6	Ashley	Taylor	403-555-1006	Picture Butte	AB	3	12.0	Edit | Delete
1	James	Wilson	403-555-9999	Lethbridge	AB	2	8.5	Edit | Delete

Figure 7: Customer Form

JonesAuto Home Forms Reports Queries

Add New Employee

First Name:

Last Name:

Phone: 403-555-1234

Role: salesperson

Add Employee

Current Employees

ID	First Name	Last Name	Phone	Role	Action
6	Ryan	Carter	403-555-0301	both	Edit | Delete
2	Dave	Richards	403-555-0102	buyer	Edit | Delete
1	Mike	Thompson	403-555-0101	buyer	Edit | Delete
5	Lisa	Blackwood	403-555-0203	salesperson	Edit | Delete
3	Sarah	Chen	403-555-0201	salesperson	Edit | Delete
4	Jake	Morrison	403-555-0202	salesperson	Edit | Delete
7	New	Tester	403-555-1111	salesperson	Edit | Delete

Figure 8: Employee Form

JonesAuto Home Forms Reports Queries

Current Inventory

Filter by Make: Filter

Showing: All Makes

Make	Model	Year	Color	Miles	Condition	Book Price	Price Paid	Repair Costs	Total Cost	Action
Chevrolet	Malibu	2017	Black	110,000	Fair	$12,500.00	$8,200.00	$270.00	$8,470.00	Sell
Chrysler	Pacifica	2020	White	62,000	Good	$22,000.00	$17,500.00	$0.00	$17,500.00	Sell
Ford	Focus	2016	Blue	135,000	Fair	$7,200.00	$4,500.00	$380.00	$4,880.00	Sell
Honda	Civic	2020	Blue	45,000	Good	$17,200.00	$13,500.00	$0.00	$13,500.00	Sell
Honda	CR-V	2019	Black	71,000	Good	$22,500.00	$17,800.00	$170.00	$17,970.00	Sell
Hyundai	Elantra	2018	White	88,000	Fair	$11,800.00	$7,500.00	$625.00	$8,125.00	Sell
Kia	Forte	2021	Silver	29,000	Excellent	$16,200.00	$12,800.00	$0.00	$12,800.00	Sell
Mazda	3	2019	Red	58,000	Good	$16,500.00	$12,500.00	$310.00	$12,810.00	Sell
Nissan	Altima	2019	Grey	67,000	Good	$15,900.00	$11,800.00	$0.00	$11,800.00	Sell
Subaru	Outback	2018	Green	93,000	Good	$19,800.00	$15,500.00	$280.00	$15,780.00	Sell
Toyota	Camry	2019	Silver	78,000	Good	$18,500.00	$14,200.00	$470.00	$14,670.00	Sell
Toyota	Tacoma	2020	White	47,000	Good	$30,200.00	$24,800.00	$95.00	$24,895.00	Sell

12 vehicle(s) found.

Figure 9: Current Inventory Report

JonesAuto Home Forms Reports Queries

Sales Report

From: mm/dd/yyyy To: mm/dd/yyyy Filter

Date	Vehicle	Customer	Salesperson	Sale Price	Cost	Repairs	Profit	Action
2026-03-15	2020 Toyota Corolla	James Wilson	Jake Morrison	$19,500.00	$14,200.00	$0.00	**$5,300.00**	Add Warranty
2025-10-25	2019 Dodge Ram 1500	Kevin Nguyen	Jake Morrison	$32,500.00	$22,000.00	$280.00	**$10,220.00**	Add Warranty
2025-10-15	2021 Toyota RAV4	Emily Brown	Jake Morrison	$29,500.00	$21,500.00	$90.00	**$7,910.00**	Add Warranty
2025-09-30	2017 Honda Accord	Ashley Taylor	Sarah Chen	$16,800.00	$10,200.00	$190.00	**$6,410.00**	Add Warranty
2025-09-20	2020 Ford Escape	Robert Garcia	Sarah Chen	$22,800.00	$15,200.00	$470.00	**$7,130.00**	Add Warranty
2025-08-25	2018 Ford F-150	James Wilson	Sarah Chen	$26,500.00	$18,500.00	$710.00	**$7,290.00**	Add Warranty
2025-08-10	2017 Chevrolet Cruze	Maria Santos	Lisa Blackwood	$11,200.00	$5,200.00	$675.00	**$5,325.00**	Add Warranty
Totals				**$158,800.00**	**$106,800.00**	**$2,415.00**	**$49,585.00**	

Figure 10: Sales Report

JonesAuto Home Forms Reports Queries

Payment History

Customer: Wilson, James

Vehicle	Due Date	Paid Date	Amount	Bank Account	Days Late	Status
2018 Ford F-150	2025-09-25	2025-09-23	$350.00	TD-4521	-2	On Time
2018 Ford F-150	**2025-10-25**	**2025-10-28**	**$350.00**	**TD-4521**	**3**	**LATE**
2018 Ford F-150	2025-11-25	2025-11-24	$350.00	TD-4521	-1	On Time
2018 Ford F-150	**2025-12-25**	**2025-12-30**	**$350.00**	**TD-4521**	**5**	**LATE**
2018 Ford F-150	2026-01-25	2026-01-22	$350.00	TD-4521	-3	On Time
2020 Toyota Corolla	2026-04-15	2026-04-13	$300.00	TD-4521	-2	On Time

Record Payment for this Customer

Figure 11: Payment History Report

JonesAuto Home Forms Reports Queries

Repair Cost Summary

Vehicle	Purchase Date	Problem #	Description	Est. Cost	Actual Cost	Difference
2019 Toyota Camry	2025-08-15	1	Scratch on rear bumper	$150.00	$120.00	$-30.00
2019 Toyota Camry	2025-08-15	2	Dent on driver door	$300.00	$350.00	+$50.00
2018 Ford F-150	2025-07-20	1	Cracked windshield	$400.00	$380.00	$-20.00
2018 Ford F-150	2025-07-20	2	Rust spot on wheel well	$200.00	$250.00	+$50.00
2018 Ford F-150	2025-07-20	3	Interior stain on back seat	$100.00	$80.00	$-20.00
2017 Chevrolet Malibu	2025-10-10	1	Paint chip on hood	$150.00	$160.00	+$10.00
2017 Chevrolet Malibu	2025-10-10	2	Broken side mirror	$120.00	$110.00	$-10.00
2021 Toyota RAV4	2025-09-25	1	Minor scratch on fender	$100.00	$90.00	$-10.00
2020 Ford Escape	2025-08-30	1	Dent on rear quarter panel	$250.00	$280.00	+$30.00
2020 Ford Escape	2025-08-30	2	Worn brake pads	$200.00	$190.00	$-10.00
2018 Hyundai Elantra	2025-10-28	1	Broken tail light	$80.00	$75.00	$-5.00
2018 Hyundai Elantra	2025-10-28	2	Seat tear on passenger side	$150.00	$200.00	+$50.00
2018 Hyundai Elantra	2025-10-28	3	AC not blowing cold	$300.00	$350.00	+$50.00
2019 Honda CR-V	2025-11-15	1	Scratch on passenger door	$180.00	$170.00	$-10.00
2017 Chevrolet Cruze	2025-07-05	1	Multiple body dents	$500.00	$620.00	+$120.00
2017 Chevrolet Cruze	2025-07-05	2	Headlight foggy	$60.00	$55.00	$-5.00
2016 Ford Focus	2025-11-20	1	Bumper crack	$200.00	$180.00	$-20.00
2016 Ford Focus	2025-11-20	2	Power window not working	$150.00	$200.00	+$50.00
2019 Dodge Ram 1500	2025-09-18	1	Rust on undercarriage	$300.00	$280.00	$-20.00
2018 Subaru Outback	2025-10-05	1	Paint fading on roof	$250.00	$240.00	$-10.00
2018 Subaru Outback	2025-10-05	2	Door handle loose	$50.00	$40.00	$-10.00
2017 Honda Accord	2025-08-12	1	Dent on trunk	$200.00	$190.00	$-10.00
2020 Toyota Tacoma	2026-04-01	1	Chip in windshield	$100.00	$95.00	$-5.00
2019 Mazda 3	2026-04-03	1	Scratch on hood	$120.00	$130.00	+$10.00
2019 Mazda 3	2026-04-03	2	Interior needs detailing	$200.00	$180.00	$-20.00

Figure 12: Repair Summary Report

JonesAuto Home Forms Reports Queries

Active Warranties

Red = Expired | Yellow = Expiring within 30 days

Customer	Vehicle	Type	Start Date	Length	Expiry Date	Days Remaining	Cost	Deductible	Items Covered	Action
Robert Garcia	2020 Ford Escape	Drive-Train	2025-09-20	7 months	2026-04-20	13	$900.00	$100.00	Engine, transmission, differential	Add Warranty
James Wilson	2018 Ford F-150	Drive-Train	2025-08-25	8 months	2026-04-25	18	$800.00	$100.00	Engine, transmission, differential, drive shaft	Add Warranty
James Wilson	2018 Ford F-150	Exterior	2025-08-25	9 months	2026-05-25	48	$400.00	$50.00	Paint, body panels, bumpers	Add Warranty
James Wilson	2020 Toyota Corolla	Electrical	2026-03-15	6 months	2026-09-15	161	$300.00	$50.00	Alternator, starter, wiring harness, battery	Add Warranty
Emily Brown	2021 Toyota RAV4	Exterior	2025-10-15	12 months	2026-10-15	191	$600.00	$50.00	Paint, body panels, bumpers, mirrors	Add Warranty
Kevin Nguyen	2019 Dodge Ram 1500	Interior	2025-10-25	12 months	2026-10-25	201	$400.00	$100.00	Seats, dashboard, carpet, headliner	Add Warranty
James Wilson	2020 Toyota Corolla	Drive-Train	2026-03-15	12 months	2027-03-15	342	$500.00	$75.00	Engine, transmission, differential	Add Warranty
Emily Brown	2021 Toyota RAV4	Drive-Train	2025-10-15	24 months	2027-10-15	556	$1,200.00	$75.00	Engine, transmission, differential, drive shaft, CV joints	Add Warranty
Kevin Nguyen	2019 Dodge Ram 1500	Drive-Train	2025-10-25	24 months	2027-10-25	566	$1,500.00	$50.00	Engine, transmission, differential, drive shaft, transfer case	Add Warranty
Kevin Nguyen	2019 Dodge Ram 1500	Exterior	2025-10-25	24 months	2027-10-25	566	$500.00	$75.00	Paint, body panels	Add Warranty

Figure 13: Warranty Report

JonesAuto Home Forms Reports Queries

Late Payment Customers

Customer	Phone	Total Payments	Late Payments	Avg Days Late	Action
Robert Garcia	403-555-1003	5	5	15.2	Record Payment
Ashley Taylor	403-555-1006	4	3	12.0	Record Payment
James Wilson	403-555-9999	6	2	8.5	Record Payment
Maria Santos	403-555-1004	4	1	3.0	Record Payment

Figure 14: Late Payments Report

JonesAuto Home Forms Reports Queries

Business Queries

7. What is our profit on each sale? (Advanced) Run Query

What is our profit on each sale?

SQL:

```
SELECT v.make, v.model, v.year, s.sale_price, p.price_paid,
       (SELECT COALESCE(SUM(r.actual_cost),0) FROM repairs r
        WHERE r.purchase_id = p.purchase_id) as repair_costs,
       (s.sale_price - p.price_paid -
        (SELECT COALESCE(SUM(r.actual_cost),0) FROM repairs r
         WHERE r.purchase_id = p.purchase_id)) as profit
FROM sales s
JOIN vehicles v ON s.vehicle_id = v.vehicle_id
JOIN purchases p ON v.vehicle_id = p.vehicle_id
WHERE s.is_active = 1
ORDER BY profit DESC
```

7 row(s) returned

make	model	year	sale_price	price_paid	repair_costs	profit
Dodge	Ram 1500	2019	32500.00	22000.00	280.00	10220.00
Toyota	RAV4	2021	29500.00	21500.00	90.00	7910.00
Ford	F-150	2018	26500.00	18500.00	710.00	7290.00
Ford	Escape	2020	22800.00	15200.00	470.00	7130.00
Honda	Accord	2017	16800.00	10200.00	190.00	6410.00
Chevrolet	Cruze	2017	11200.00	5200.00	675.00	5325.00
Toyota	Corolla	2020	19500.00	14200.00	0.00	5300.00

Figure 15: Query 7 Results: Profit per Sale